

Mata Kuliah : **Praktikum Aplikasi Berbasis Platform**
Pertemuan : **11**
Nama : **Naufal Thoriq Muzhaffar**
NIM : **2311102078**

1. Pendahuluan & Dokumentasi

1.1 Tampilan Daftar Tugas & Penambahan Tugas Baru

Aventuria Store adalah aplikasi mobile e-commerce bertema *vintage fantasy* yang dibangun menggunakan Flutter 3.41.7 dengan pola arsitektur **Cubit** dari paket flutter_bloc. Aplikasi ini mengimplementasikan prinsip *separation of concerns* secara ketat melalui pemisahan lapisan models/, cubit/, screens/, widgets/, dan theme/ — memastikan tidak ada logika bisnis yang bocor ke lapisan UI.

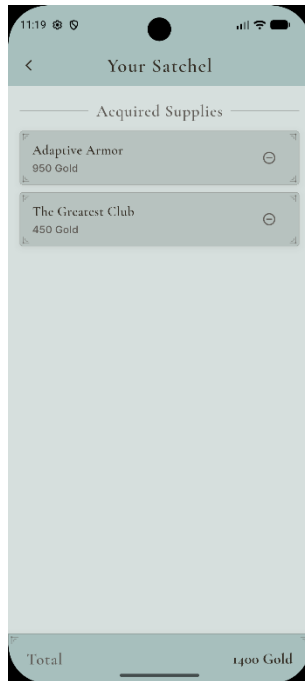
Sistem desain visual ditegakkan secara terpusat melalui kelas AppColors dan AppTextStyles yang mendefinisikan palet warna *vintage* (#322B22 sebagai *dark accent*, #A7BFBD sebagai *header background*, #605C57 sebagai *heading*, #85817E sebagai *body text*, #B7C2BF sebagai *card background*), serta tipografi kustom *Cormorant Garamond* untuk elemen dekoratif dan *Inter* untuk elemen fungsional. Ornamen sudut dekoratif pada setiap panel digambar secara programatik menggunakan CustomPainter, mengeliminasi ketergantungan pada aset gambar eksternal untuk elemen struktural UI.

1.2 Dokumentasi Antarmuka (UI)



Layar 1 — Product List Screen

Layar utama merender daftar produk menggunakan ListView.separated dengan _ProductCard sebagai item builder, di mana setiap kartu membungkus DecorativeCard — widget *reusable* yang menggunakan Stack untuk melapisi konten dengan empat CustomPaint widget (_OrnamentPainter) yang menggambar ornamen *bezier scroll* di setiap sudut secara terprogram. Widget _CartBadge pada AppBar terhubung langsung ke CartCubit melalui BlocBuilder dengan buildWhen yang terisolasi, sehingga badge jumlah item diperbarui secara reaktif tanpa men-trigger rebuild pada widget-widget lain di layar yang sama.



Layar 2 — Cart Screen (Your Satchel)

Layar satchel menampilkan daftar item aktif menggunakan `ListView.separated` yang diisi dari `state.items` hasil `BlocBuilder<CartCubit, CartState>`, di mana setiap baris dibungkus `DecorativeCard` dengan `IconButton` hapus yang memanggil `removeFromCart` secara langsung via `context.read<CartCubit>()`. Panel total di bagian bawah layar adalah `DecorativeCard` bertipe *full-width* yang membaca getter `state.totalPrice` — sebuah nilai terhitung (*computed value*) dari akumulasi `items.fold()` di dalam `CartState` — sehingga total selalu sinkron dengan isi keranjang tanpa logika kalkulasi tambahan di lapisan UI.

2. Hasil Implementasi

2.1 Analisis Arsitektur Cubit

Cubit dipilih sebagai solusi manajemen state atas pertimbangan berikut dibandingkan `setState` native Flutter:

Immutabilitas State. `CartState` didefinisikan sebagai kelas `@immutable` dengan `copyWith()` sebagai satu-satunya mekanisme mutasi. Setiap perubahan menghasilkan instance baru, menjamin tidak ada referensi *shared mutable state* antar widget — sebuah jaminan yang tidak dapat diberikan oleh `setState` tanpa disiplin manual dari developer.

Prediktabilitas & Testabilitas. Seluruh transisi state dikonsolidasikan dalam satu kelas (`CartCubit`) dan diaktifkan eksklusif melalui `emit()`. Hal ini membuat alur data bersifat *unidirectional* dan deterministik: UI hanya mendeskripsikan tampilan berdasarkan state, tidak pernah memodifikasinya secara langsung.

Boilerplate Minimal. Dibandingkan arsitektur BLoC penuh yang memerlukan definisi Event class terpisah, Cubit mengekspos metode publik secara langsung (`addToCart`, `removeFromCart`), memangkas boilerplate secara signifikan tanpa mengorbankan prinsip enkapsulasi logika bisnis.

2.2 Struktur & Alur Data

Alur data dalam proyek bersifat *top-down* dan *unidirectional* melewati lapisan berikut:

- **theme/** — `AppColors` dan `AppTextStyles` didefinisikan sebagai abstract final class dengan member static const, dikonsumsi langsung oleh seluruh widget tanpa perlu `Theme.of(context)` berulang. `appTheme` (objek `ThemeData`) diinisialisasi satu kali di `main.dart`.
- **models/** — `Product` adalah *immutable data class* dengan operator `==` dan `hashCode` berbasis id. Konstanta `kProducts` (`const List<Product>`) berfungsi sebagai sumber data statis tunggal untuk `ProductListScreen`.
- **cubit/** — `CartCubit` mewarisi `Cubit<CartState>`. `CartState` didefinisikan sebagai part file untuk menjaga kohesi logis sekaligus memisahkan definisi state secara fisik. Setiap

mutasi daftar menggunakan `List.from()` untuk menghasilkan koleksi baru yang terlepas dari referensi state sebelumnya, memenuhi kontrak immutabilitas.

- **widgets/** — `DecorativeCard` dan `ProductArtwork` merupakan *pure presentational widgets* tanpa akses ke `BuildContext` Cubit, memastikan komponen ini sepenuhnya portabel dan dapat diuji secara terisolasi.
- **screens/** — `BlocProvider<CartCubit>` diinstansiasi di `main.dart` (root widget tree), menjamin single instance `CartCubit` yang persisten sepanjang siklus hidup aplikasi. Konsumsi di `ProductListScreen` dan `CartScreen` dilakukan melalui `BlocBuilder` dengan parameter `buildWhen` untuk membatasi scope *rebuild* pada subtree widget yang relevan saja.

2.3 Snippet Kode Utama

Snippet 1 — Metode `addToCart` dan `removeFromCart` pada `CartCubit`

```
class CartCubit extends Cubit<CartState> {
  CartCubit() : super(const CartState());

  void addToCart(Product product) {
    // Guard duplikasi menggunakan Product.operator== berbasis id
    if (state.items.contains(product)) return;
    emit(state.copyWith(items:
List.from(state.items)..add(product)));
  }

  void removeFromCart(Product product) {
    emit(state.copyWith(
      items: List.from(state.items)..remove(product),
    ));
  }
}
```

`List.from()` digunakan secara eksplisit untuk memutus referensi dari list state sebelumnya sebelum operasi mutasi, memastikan `CartState` lama dan baru adalah objek yang benar-benar terpisah di memori.

Snippet 2 — `BlocBuilder` untuk Badge Keranjang Real-Time

```
class _CartBadge extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return BlocBuilder<CartCubit, CartState>(
      // buildWhen membatasi rebuild HANYA saat jumlah item
      berubah,
      // mengeliminasi rebuild yang dipicu oleh perubahan state
      lain.
      buildWhen: (previous, current) =>
```

```

        previous.items.length != current.items.length,
builder: (context, state) {
    return Stack(
        alignment: Alignment.center,
        children: [
            IconButton(
                icon: const Icon(Icons.shopping_bag_outlined),
                onPressed: () => Navigator.push(
                    context,
                    MaterialPageRoute(builder: (_) => const
CartScreen()),
                ),
            ),
            if (state.items.isNotEmpty)
                Positioned(
                    top: 6, right: 6,
                    child: Container(
                        padding: const EdgeInsets.all(3),
                        decoration: const BoxDecoration(
                            color: AppColors.heading,
                            shape: BoxShape.circle,
                        ),
                        child: Text(
                            '${state.items.length}',
                            style: AppTextStyles.badge,
                        ),
                    ),
                ),
        ],
    );
},
);
}
}

```

Parameter `buildWhen` merupakan optimasi kritis: `_CartBadge` hanya di-*rebuild* ketika `items.length` berubah, bukan pada setiap `emit()` yang mungkin terjadi. Ini memastikan performa `AppBar` tetap optimal meskipun frekuensi perubahan state tinggi.